



Software Requirements Specification (SRS)

Project: StackAI

Version: 1.0

Date: August 2025

Prepared by: N Bhanu Durga Prasad

1. Introduction

1.1 Purpose

The purpose of this document is to define the **Software Requirement Specifications (SRS)** for **StackAI**, an all-in-one AI-powered productivity platform. StackAI integrates content generation, image editing, resume creation/review, and a community feature, providing users with a seamless multi-tool AI experience.

This document is intended for:

- **Developers** (Frontend + Backend teams)
 - **Testers** (QA, Integration)
 - **Project Stakeholders** (Students, Recruiters, Users)
-

1.2 Scope

StackAI provides a single platform that combines multiple AI-powered utilities:

- **Content generation** (Articles, Blogs, Resume reviews)
- **Image generation and editing** (AI images, background removal, object removal)
- **Resume Builder**
- **Community Sharing** of AI-generated outputs

The platform ensures **secure authentication, subscription billing, and cloud-hosted image services**, with a responsive frontend and scalable backend.

- **Frontend:** React.js, Clerk authentication, API integrations
 - **Backend:** Node.js, Express.js, PostgreSQL (Neon DB), Gemini API, Clipdrop API, Cloudinary
 - **Users:** Students, professionals, job seekers, creators
-

1.3 Definitions, Acronyms, and Abbreviations

- **API** – Application Programming Interface
 - **JWT** – JSON Web Token
 - **SaaS** – Software as a Service
 - **DB** – Database
 - **UI/UX** – User Interface / User Experience
-

1.4 References

- Gemini API Docs: <https://ai.google.dev>
 - Clipdrop API Docs: <https://clipdrop.co>
 - Clerk Auth Docs: <https://clerk.com/docs>
 - Neon DB: <https://neon.tech>
-

2. Overall Description

2.1 Product Perspective

StackAI is a **web-based SaaS platform** with **client-server architecture**.

- **Frontend (Client):** Provides the UI, tool dashboard, and community display.
 - **Backend (Server):** Handles API requests, authentication, billing, and database operations.
 - **External APIs:** Gemini, Clipdrop, Clerk, Cloudinary.
 - **Database:** Neon DB (PostgreSQL).
-

2.2 Product Functions

1. Authentication & Billing

- User registration/login via Clerk
- Token-based secure session handling
- Subscription billing integration

2. AI Tools

- Article/blog content generation (Gemini API)
- Resume review (Gemini API)
- Resume creation (Form-based input → Resume template)
- Image generation (Clipdrop API)
- Background/Object removal (Clipdrop API)

3. Community Section

- Upload AI-generated images (publicly)
- Fetch and display community-shared images

4. User Profile

- Manage subscription plan
 - Access saved/generated content
-

2.3 User Characteristics

- **General Users:** Students, creators, professionals looking for quick AI tools.
 - **Admin Users:** Manage community content, monitor billing and analytics.
-

2.4 Constraints

- Internet connection required (SaaS product).
 - External API rate limits (Gemini, Clipdrop).
 - Free-tier restrictions (limited tools without subscription).
-

2.5 Assumptions & Dependencies

- Clerk is available for authentication & billing.
 - Gemini and Clipdrop APIs maintain uptime.
 - Cloudinary reliably stores and serves images.
-

3. Specific Requirements

3.1 Functional Requirements

Authentication & Authorization

- FR1: Users must be able to register and log in using Clerk.
- FR2: Users must authenticate using JWT tokens.
- FR3: Only subscribed users can access premium tools.

Content Generation (Gemini API)

- FR4: System must generate articles, blogs, and resume reviews.
- FR5: Input fields must allow user-provided prompts.

Resume Builder

- FR6: Users must be able to fill a form and generate resumes.
- FR7: System must provide a downloadable PDF option.

Image Tools (Clipdrop API)

- FR8: Generate AI images from prompts.
- FR9: Remove background from uploaded images.
- FR10: Remove selected objects from uploaded images.

Community Section

- FR11: Users can publish images to community if marked "public".
- FR12: Users can view and interact with community images.

Database & Storage

- FR13: Neon DB stores user data, subscription status, and activity logs.
- FR14: Cloudinary stores and retrieves AI-generated images.

3.2 Non-Functional Requirements

- **Performance:** Responses must be under 3 seconds for API calls.
 - **Scalability:** System must handle 10,000+ concurrent users.
 - **Availability:** 99.5% uptime target.
 - **Security:** JWT tokens, password hashing (bcrypt), Clerk for identity.
 - **Usability:** Responsive UI across devices.
 - **Maintainability:** Code structured with component-based (frontend) and MVC (backend).
-

4. System Models

4.1 Use Case Diagram

Actors:

- User (Student/Professional)
- Admin

Main Use Cases:

- Login/Register
- Generate Content
- Generate/Modify Images
- Build Resume
- Share/View Community Content

User -----> [Login/Register]
| [Generate Content]
| [Generate Image / Edit Image]
| [Build Resume]
| [Share Community Content]
Admin -----> [Manage Community Content]

4.2 Data Flow (High-Level)

1. User logs in → Clerk generates JWT → Stored in LocalStorage.
 2. User selects AI tool → Request sent to Backend.
 3. Backend validates JWT → Calls external API (Gemini/Clipdrop).
 4. API response stored in DB (if needed).
 5. Frontend renders output → User may save/share.
-

5. External Interface Requirements

5.1 User Interface (UI/UX)

- Dashboard with AI tools (cards/grid)
- Resume form builder
- Image upload/generation tool with preview
- Community gallery

5.2 API Interfaces

- Gemini API → [/content/generate](#)
- Clipdrop API → [/image/generate](#), [/image/remove-bg](#)
- Clerk → [/auth/login](#), [/auth/register](#)

- Cloudinary → Image upload & fetch

5.3 Hardware Interfaces

- Client: Modern browser, 2GB RAM+
 - Server: Node.js runtime, 2GB RAM+, PostgreSQL DB
-

6. Appendix

- **Future Enhancements:**
 - AI-powered interview prep tool
 - Multi-language support
 - Mobile app version
-